

Solaris Deployment Guide

Complete instructions for running Solaris locally or deploying to cloud platforms.

Getting the code

From Abacus.AI Agent

1. Click **Files icon** (📁) in top-right
2. Navigate to `/home/ubuntu/`
3. Download `solaris-build-YYYYMMDD.tar.gz`
4. Extract: `tar -xzf solaris-build-*.tar.gz`

Or download `/home/ubuntu/luca-passport/` directory directly.

Local setup

Prerequisites

- **Node.js** 18+ (`node --version`)
- **PostgreSQL** 14+ (`psql --version`)

1. Database setup

```
# Start PostgreSQL (macOS/Homebrew example)
brew services start postgresql@14

# Create database and user
psql postgres -c "CREATE USER luca_user WITH PASSWORD 'luca_dev_2026';"
psql postgres -c "CREATE DATABASE luca_passport OWNER luca_user;"

# Apply schema
cd luca-passport/backend
psql "postgresql://luca_user:luca_dev_2026@localhost:5432/luca_passport" -f schema.sql
psql "postgresql://luca_user:luca_dev_2026@localhost:5432/luca_passport" -f schema_solaris.sql

# Seed demo data
node seed_solaris.js
```

2. Backend

```
cd backend
npm install

# Create .env (or edit existing)
cat > .env << 'EOF'
DATABASE_URL=postgresql://luca_user:luca_dev_2026@localhost:5432/luca_passport
JWT_SECRET=luca_dev_secret_2026_sovereign_health
PORT=5000
NODE_ENV=development
EOF

# Start server
node src/server.js
# Should see: ✓ LUCA Passport Backend running on port 5000
```

3. Frontend

```
# In project root
npm install

# Start dev server
npx vite
# Opens at http://localhost:3000
```

4. Test

- Open <http://localhost:3000>
- Login: `sarah@solaris.health` / `demo123`



Cloud deployments



Railway (recommended for MVP)

One-click Postgres + auto-deploy from GitHub

1. **Push to GitHub** (if not already):

```
bash
git remote add origin https://github.com/YOUR_USERNAME/solaris-health.git
git push -u origin main
```

2. **Deploy:**

- Go to railway.app (<https://railway.app>)
- “New Project” → “Deploy from GitHub”
- Select your repo
- Railway auto-detects Node.js

3. **Add Postgres:**

- Click “New” → “Database” → “PostgreSQL”
- Railway creates `DATABASE_URL` automatically

4. **Apply schema:**

```
bash
```

```
# Get Railway DB connection string from dashboard
export DATABASE_URL="postgresql://..."
psql "$DATABASE_URL" -f backend/schema.sql
psql "$DATABASE_URL" -f backend/schema_solaris.sql
cd backend && node seed_solaris.js
```

5. **Environment variables** (Railway dashboard):

```
DATABASE_URL=(auto-generated)
JWT_SECRET=your_production_secret_here
NODE_ENV=production
PORT=5000
```

6. **Deploy frontend:**

Railway auto-deploys both backend and frontend if you add a `railway.toml` :

```
``toml
```

```
[build]
```

```
builder = "nixpacks"
```

```
[deploy]
```

```
startCommand = "npm run dev"
```

```
```
```

**Cost:** Free tier includes 500 hours/month + 5GB storage.



## Render

#### 1. **Create Web Service** (backend):

- Connect GitHub repo
- Root directory: `backend`
- Build: `npm install`
- Start: `node src/server.js`

#### 2. **Create PostgreSQL:**

- "New" → "PostgreSQL"
- Copy internal connection string

#### 3. **Environment variables:**

```
DATABASE_URL=(from Render Postgres)
JWT_SECRET=your_production_secret
NODE_ENV=production
```

#### 4. **Apply schema** (via Render Shell):

```
bash
psql $DATABASE_URL -f schema.sql
psql $DATABASE_URL -f schema_solaris.sql
node seed_solaris.js
```

#### 5. **Create Static Site** (frontend):

- Build: `npm install && npm run build`
  - Publish: `dist`
  - Add environment variable:
- ```
VITE_API_URL=https://your-backend.onrender.com/api
```

Cost: Free tier available (spins down after inactivity).

Fly.io

1. Install Fly CLI:

```
bash
curl -L https://fly.io/install.sh | sh
fly auth login
```

2. Deploy Postgres:

```
bash
fly postgres create --name solaris-db --region ord
# Save connection string
```

3. Deploy backend:

```
```bash
cd backend
fly launch -name solaris-api
Edit fly.toml:
[env]
PORT = "5000"
NODE_ENV = "production"

fly secrets set DATABASE_URL="postgres://..." JWT_SECRET="..."
fly deploy
```
```

1. Apply schema:

```
bash
fly postgres connect -a solaris-db
# Then run schema.sql, schema_solaris.sql, seed
```

2. Deploy frontend:

```
bash
cd ..
fly launch --name solaris-app
# Add to fly.toml:
# [[env]]
# VITE_API_URL = "https://solaris-api.fly.dev/api"
fly deploy
```

Cost: Free allowances: 3 VMs + 3GB storage.

Vercel + Supabase (alternative)

Frontend on Vercel, Postgres on Supabase

1. Supabase (free Postgres):

- supabase.com (<https://supabase.com>) → New Project

- Copy connection string (pooler mode)
- Run schema via SQL Editor or `psql`

2. **Vercel** (frontend):

- vercel.com (<https://vercel.com>) → Import Git
- Root: `./`
- Build: `npm run build`
- Output: `dist`
- Environment:
`VITE_API_URL=your_backend_url/api`

3. **Backend** (host separately on Render/Railway/Fly)

Cost: Both have generous free tiers.

Production checklist

Before going live:

- ☐ Change `JWT_SECRET` to a strong random value
- ☐ Change all database passwords
- ☐ Update CORS origins in `backend/src/server.js`
- ☐ Set `NODE_ENV=production`
- ☐ Enable HTTPS (Railway/Render/Fly auto-provide)
- ☐ Remove demo users or change passwords
- ☐ Set up database backups
- ☐ Configure environment-specific `VITE_API_URL` in frontend

Sovereign deployment (advanced)

For maximum sovereignty:

Self-hosted VPS (DigitalOcean, Hetzner, Linode)

```
# SSH into server
ssh root@your-server-ip

# Install dependencies
apt update && apt install -y nodejs npm postgresql nginx certbot

# Clone repo
git clone https://github.com/you/solaris-health.git
cd solaris-health

# Set up Postgres, install, start services
# Configure Nginx reverse proxy
# Get SSL cert via certbot
```

Docker Compose

```
# docker-compose.yml (create this)
services:
  db:
    image: postgres:15
    environment:
      POSTGRES_USER: luca_user
      POSTGRES_PASSWORD: ${DB_PASSWORD}
      POSTGRES_DB: luca_passport
    volumes:
      - pgdata:/var/lib/postgresql/data

  backend:
    build: ./backend
    environment:
      DATABASE_URL: postgresql://luca_user:${DB_PASSWORD}@db:5432/luca_passport
      JWT_SECRET: ${JWT_SECRET}
    depends_on:
      - db

  frontend:
    build: .
    environment:
      VITE_API_URL: /api
    ports:
      - "3000:3000"

volumes:
  pgdata:
```

Run: `docker-compose up -d`

Support

Questions? Issues deploying?

- Check `backend/logs` for errors
- Verify `DATABASE_URL` format: `postgresql://user:pass@host:port/db`
- Ensure all schema files applied in order
- Run seed script after schema

Environment variables required:

- `DATABASE_URL` (Postgres connection)
- `JWT_SECRET` (any random string, 32+ chars recommended)
- `PORT` (5000 for backend, 3000 for frontend)
- `NODE_ENV` (development or production)